

Code-Layout, Readability and Reusability

Date	19 June 2026
Team ID	
Project Name	EcoTrack - Personal Carbon AI Footprint Tracker
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

Well-structured code is essential for maintainability and collaboration. Document the standards, patterns, and principles your team followed to ensure the codebase is clean, readable, and highly reusable. Provide specific examples of how these practices were implemented in your project.

Code Quality & Structure Details

S.No	Quality Principle	Implementation Details / Description	Specific Project Examples (Files/Folders)
1	Directory Structure	Clean modular component separation segregating React UI views, Flask REST controllers, and Firebase services.	src/components/ src/pages/ backend/app.py
2	Naming Conventions	Strict adherence to PascalCase for React components, camelCase for JavaScript variables, & snake_case for Python Flask routes.	HabitLogger.jsx, calculateEmissions(), get_user_logs()
3	Readability & Docs	Comprehensive JSDoc and Python docstrings describing function signatures, input parameters, & return types.	src/utils/calc.js, backend/api/routes.py
4	Modularity (DRY)	Reusable glassmorphism card components, custom React hooks for Firebase auth, and shared emission calculation helpers.	src/components/Card.jsx, src/hooks/useAuth.js

S.No	Quality Principle	Implementation Details / Description	Specific Project Examples (Files/Folders)
5	Formatting & Linting	Automated code formatting using Prettier and static code linting via ESLint and Python PEP8 Flake8.	.eslintrc.json, .prettierrc, backend/flake8
6	Error Handling	Centralized Flask REST API error handling middleware and React Error Boundary fallbacks with toast notifications.	backend/middleware.py, src/components/ErrorModal.jsx